

YGREKY

Linux Secure by Default

Are we there yet?

Marta Rybczynska, Ygreky



Secure by default - a requirement
But this is not a Cyber Resilience Act talk

YGREKY



Official Journal
of the European Union

EN

L series

2024/2847

20.11.2024

REGULATION (EU) 2024/2847 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL

of 23 October 2024

on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) No 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act)

(Text with EEA relevance)

THE EUROPEAN PARLIAMENT AND THE COUNCIL OF THE EUROPEAN UNION,

Having regard to the Treaty on the Functioning of the European Union, and in particular Article 114 thereof,

Having regard to the proposal from the European Commission,

After transmission of the draft legislative act to the national parliaments,

Having regard to the opinion of the European Economic and Social Committee ([1]),

After consulting the Committee of the Regions,

Acting in accordance with the ordinary legislative procedure ([2]),

What we have in the Linux kernel

YGREKY

docs.kernel.org/security/index.html

Security Documentation

- Credentials in Linux
- Confidential Computing in Linux for x86 virtualization
- IMA Template Management Mechanism
- IMA Measurements Export and Delete
- Kernel Keys
- Linux Security Modules: General Security Hooks for Linux
- Linux Security Module Development
- Linux Secure Attention Key (SAK) handling
- SCTP
- Kernel Self-Protection
- SipHash - a short input PRF
- HalfSipHash - SipHash's insecure younger cousin
- Trusted Platform Module documentation
- Digital Signature Verification API
- Landlock LSM: kernel documentation
- Secrets documentation
- Integrity Policy Enforcement (IPE) - Kernel Documentation

Linux Kernel Self-Protection Project

Mission Statement

Founded in 2015, this upstream Linux kernel project starts with the premise that kernel bugs have a very long lifetime, and that the kernel must be designed in ways to protect against these flaws. We must think of security beyond fixing bugs. As a community, we already find and fix individual bugs via static checkers (compiler flags, smatch, coccinelle, coverity, CodeQL) and dynamic checkers (kernel configs, syzkaller, KASan, trinity). Those efforts are important and on-going, but if we want to protect our billion Android phones, our cars, the International Space Station, and everything else running Linux, we must get proactive defensive technologies built into the upstream Linux kernel. We need the kernel to fail safely, instead of just running safely.

These kinds of protections have existed for years in out-of-tree projects and in piles of academic papers. For various social, cultural, and technical reasons, they have not made their way into the upstream kernel, and this project seeks to change that. Our focus is on kernel self-protection, rather than kernel-supported userspace protections. The goal is to eliminate classes of bugs and eliminate methods of exploitation.

Security-related interfaces

- No New Privileges Flag
- Seccomp BPF (SECure COMPuting with filters)
- Landlock: unprivileged access control
- Linux Security Modules
- Introduction of non-executable mfd
- Speculation Control
- TEE (Trusted Execution Environment) Userspace API
- Executability check

kernel.org Added as CVE Numbering Authority (CNA)

News February 13, 2024

kernel.org is now a CVE Numbering Authority (CNA) for any vulnerabilities in the Linux kernel as listed on kernel.org, excluding end-of-life (EOL) versions.



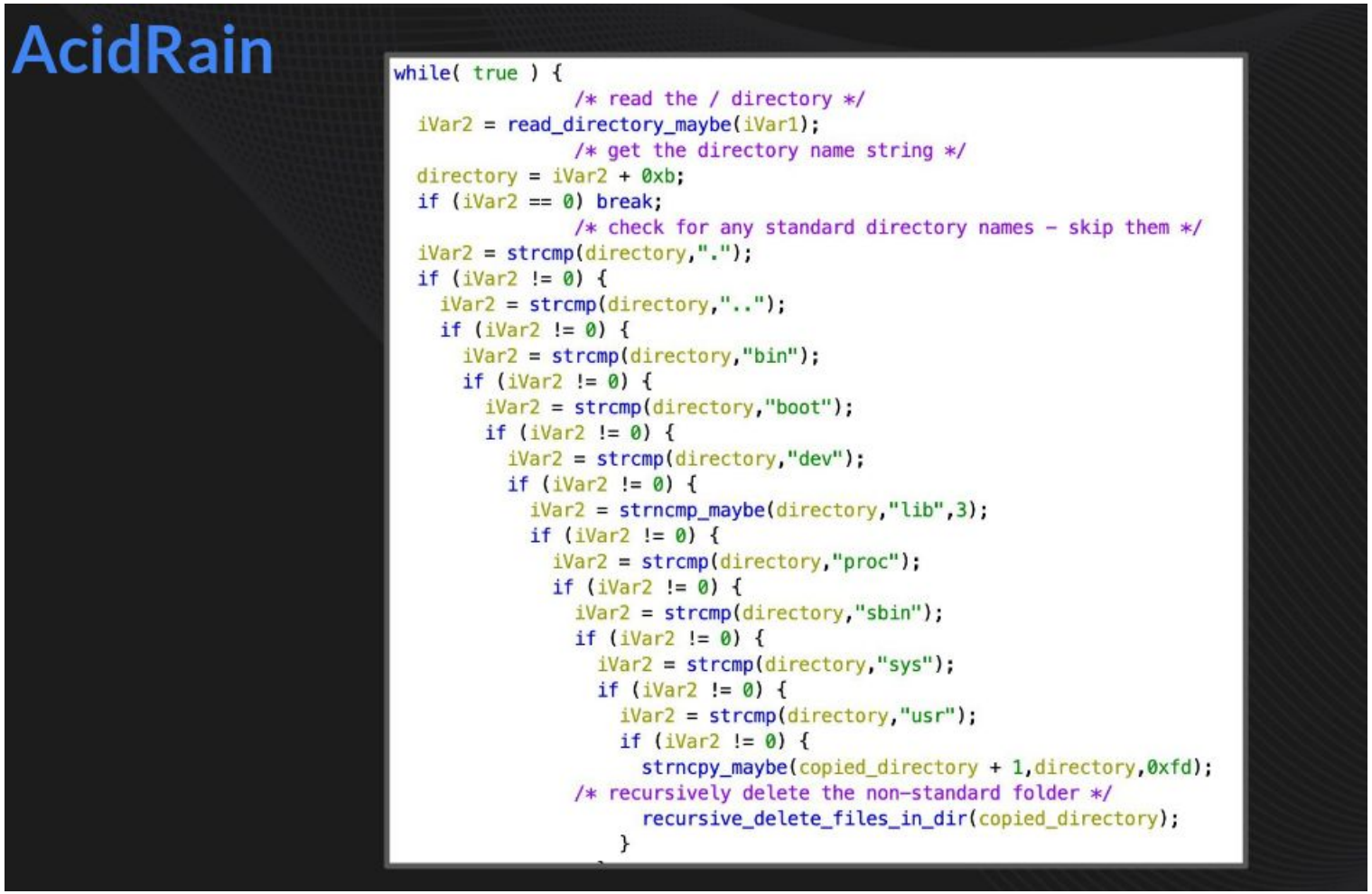
Isn’t it “done”?



At the same time, here's what we have in embedded...
An example of embedded wiper malware

YGREKY

- The slide comes from "Real 'Cyber War': Espionage, DDoS, Leaks, and Wipers in the Russian Invasion of Ukraine" by Tom Hegel and Juan Andres Guerrero-Saade, Blackhat 2022 https://i.blackhat.com/USA-22/Wednesday/US-22-Hegel-Real-Cyber-War.pdf



Why?



Some of (partial) reasons



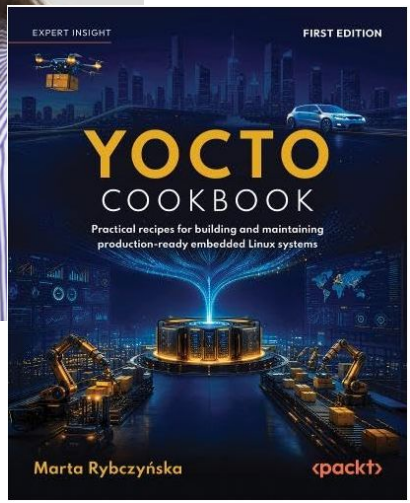
- Embedded is running old/vendor kernels
  - Often after the LTS end
  - With vendor patches
- Enabling new features can be complex
- Embedded developers aren’t kernel experts
- Time pression

Who is Marta Rybczynska?

YGREKY

- PhD in Telecommunications
  - Network security/anonymity systems
- Open source/embedded developer/architect
  - 20+ years in open source
  - Contributions to the Linux kernel, various RTOSes
  - Involved into Yocto Project security
- Founder/CEO of Ygreky
  - Consulting (processes, architecture, reviews)
  - Teaching (“Embedded Security”, cybersecurity on-site courses and webinars)
  - Contributing to CRA-related standardisation





The experiment



Research questions

1. Is it **more likely** to find an option enabled on an embedded platform if it is **enabled by default** in the kernel?
2. Are security options set **the same way** on embedded platforms?
3. Do distributions **tend to follow** security recommendations?

Kernel options testing



Linux kernel default	7.2
KSPP recommendation	Current (September 2026)
Ubuntu	24.04 (kernel 6.8.0)
Yocto Project	“Wrynose” (current LTS) (kernel 6.18.28)
OpenWRT	2025.12 (latest stable, kernel 6.12.108)

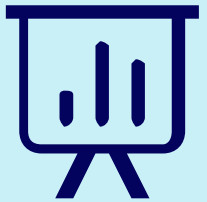
Assumptions:

- X86 architecture for all builds
- Only default settings

Tooling:

- Kernel-hardening-checker
- Custom post-processing scripts

YGREKY



Part 1: kernel configuration

Kernel options testing
CONFIG_STACKPROTECTOR_STRONG

YGREKY

Linux kernel default	**Y**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	Y
OpenWRT	**Is not set**

Notes on CONFIG_STACKPROTECTOR_STRONG:

- Adds canaries for local variables, under some conditions
- Enables -fstack-protector-strong

Kernel options testing
CONFIG_FORTIFY_SOURCE

YGREKY

Linux kernel default	**No default**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	**Is not set**
OpenWRT	Y

Notes on CONFIG_FORTIFY_SOURCE:

- Detects buffer overflows in common string-related functions

Kernel options testing
CONFIG_DEBUG_WX

YGREKY

Linux kernel default	**No default**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	**Is not set**
OpenWRT	Is not set

Notes on CONFIG_DEBUG_WX:

- Warning if any MMU W+X (write+execute) mappings are found at boot.
- One time check at boot.
- Doesn’t stop the boot.

Kernel options testing
CONFIG_RANDOMIZE_BASE

YGREKY

Linux kernel default	**Y on x86**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	Y
OpenWRT	Is not set

Notes on CONFIG_RANDOMIZE_BASE:

- Enables randomisation of the physical address at which the kernel image is decompressed and the virtual address where the kernel image is mapped
- Architecture-level switch, supported on some architectures: x86, s390, some riscv and powerpc configurations etc
- Enabled by default on some platforms: for example x86, s390

Kernel option testing
CONFIG_HARDENED_USERCOPY

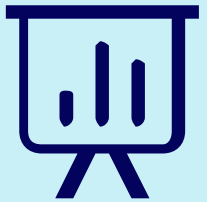
YGREKY

Linux kernel default	**No default**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	Y
OpenWRT	Is not set

Notes on CONFIG_HARDENED_USERCOPY:

- Checks for wrong memory regions when copying memory to/from the kernel in copy_to_user() and copy_from_user()

YGREKY



Part 2: distro impact

A story: Yocto Project builds on Ubuntu
Since 24.04

YGREKY

- YP uses user namespaces to remove network access from some processes
  - Uses the “unshare” system call
  - Disable downloading code from build stages other than downloading: possible malware mitigation
  - Ticket: https://bugs.launchpad.net/ubuntu/+source/apparmor/+bug/2056555



Allow bitbake to create user namespace

Bug #2056555 reported by Hains on 2024-03-08

This bug affects 57 people

Affects	Status	Importance	Assigned to
apparmor (Ubuntu)	Confirmed	Undecided	Unassigned

Bug Description

```
Occurs since an update around March 2 Ubuntu 24.04.

Bitbake is broken due to file permission problem.

Traceback (most recent call last):
  File "/home/hains/openpli-oe-core/bitbake/bin/bitbake-worker", line 268, in child
    bb.utils.disable_network(uid, gid)
  File "/home/hains/openpli-oe-core/bitbake/lib/bb/utils.py", line 1653, in disable_network
    with open("/proc/self/uid_map", "w") as f:
PermissionError: [Errno 1] Operation not permitted

Test code

with open("/proc/self/uid_map", "w") as f:
 f.write("%s %s 1" % (1000, 1000))
```

A story: Yocto Project builds on Ubuntu
Since 24.04

YGREKY

- YP uses user namespaces to remove network access from some processes
  - Uses the “unshare” system call
  - Disable downloading code from build stages other than downloading: possible malware mitigation
  - Ticket: https://bugs.launchpad.net/ubuntu/+source/apparmor/+bug/2056555
- Doesn’t work with Ubuntu apparmor profile
- No better workaround than:

```
echo 0 | sudo tee /proc/sys/kernel/apparmor_restrict_unprivileged_userns
```

Allow bitbake to create user namespace

Bug #2056555 reported by Hains on 2024-03-08

This bug affects 57 people

Affects	Status	Importance	Assigned to
apparmor (Ubuntu)	Confirmed	Undecided	Unassigned

Bug Description

```
Occurs since an update around March 2 Ubuntu 24.04.

Bitbake is broken due to file permission problem.

Traceback (most recent call last):
  File "/home/hains/openpli-oe-core/bitbake/bin/bitbake-worker", line 268, in child
    bb.utils.disable_network(uid, gid)
  File "/home/hains/openpli-oe-core/bitbake/lib/bb/utils.py", line 1653, in disable_network
    with open("/proc/self/uid_map", "w") as f:
PermissionError: [Errno 1] Operation not permitted

Test code

with open("/proc/self/uid_map", "w") as f:
 f.write("%s %s 1" % (1000, 1000))
```

Richard-purdie-1 (richard-purdie-1) wrote on 2024-09-18:

```
Unfortunately even the /**/bitbake/bin/bitbake mask doesn't always work since bitbake's server can be triggered by other commands
such as devtool or recipetool. In those cases, the glob doesn't match and you hit weird failures. It also has interesting problems
with using BB_SERVER_TIMEOUT=60 since it can remain in memory with a different execution name.

So far the only reliable solution I've found was to turn this restriction off entirely. I wanted to mention this here in case others
run into it.
```

Kernel options testing
CONFIG_SECURITY

YGREKY

Linux kernel default	**Y**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	Y
OpenWRT	Is not set

Notes on CONFIG_SECURITY:

- Enabled security modules

Kernel options testing
CONFIG_LSM

Linux kernel default	**Various (landlock,lockdown,yama,loadpin,safesetid,ipe,bpf if no DEFAULT_SECURITY_* option)**
KSPP recommendation	lockdown
Ubuntu	landlock,lockdown,yama,integrity,apparmor
Yocto Project	landlock,lockdown,yama,loadpin,safesetid,ipe,bpf
OpenWRT	Not found

Notes on CONFIG_LSM:

- Initialization order of LSMs
- Complex default setup:

```
default "landlock,lockdown,yama,loadpin,safesetid,smack,selinux,tomoyo,apparmor,ipe,bpf" if DEFAULT_SECURITY_SMACK
default "landlock,lockdown,yama,loadpin,safesetid,apparmor,selinux,smack,tomoyo,ipe,bpf" if DEFAULT_SECURITY_APPARMOR
default "landlock,lockdown,yama,loadpin,safesetid,tomoyo,ipe,bpf" if DEFAULT_SECURITY_TOMOYO
default "landlock,lockdown,yama,loadpin,safesetid,ipe,bpf" if DEFAULT_SECURITY_DAC
default "landlock,lockdown,yama,loadpin,safesetid,selinux,smack,tomoyo,apparmor,ipe,bpf"
```

Kernel options testing
CONFIG_MODULE_SIG



Linux kernel default	**No default**
KSPP recommendation	Y
Ubuntu	Y
Yocto Project	Is not set
OpenWRT	Is not set

Notes on CONFIG_MODULE_SIG:

- Check module signature when loading
- To be used with the lockdown feature of an LSM

Kernel options testing
CONFIG_DEVMEM

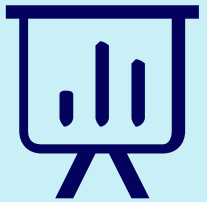


Linux kernel default	**Y**
KSPP recommendation	**Is not set**
Ubuntu	Y
Yocto Project	Y
OpenWRT	Is not set

Notes on CONFIG_DEVMEM:

- Enabled /dev/mem, that allows access to physical memory

YGREKY



Part 3: applications

A story: Docker and firewalls

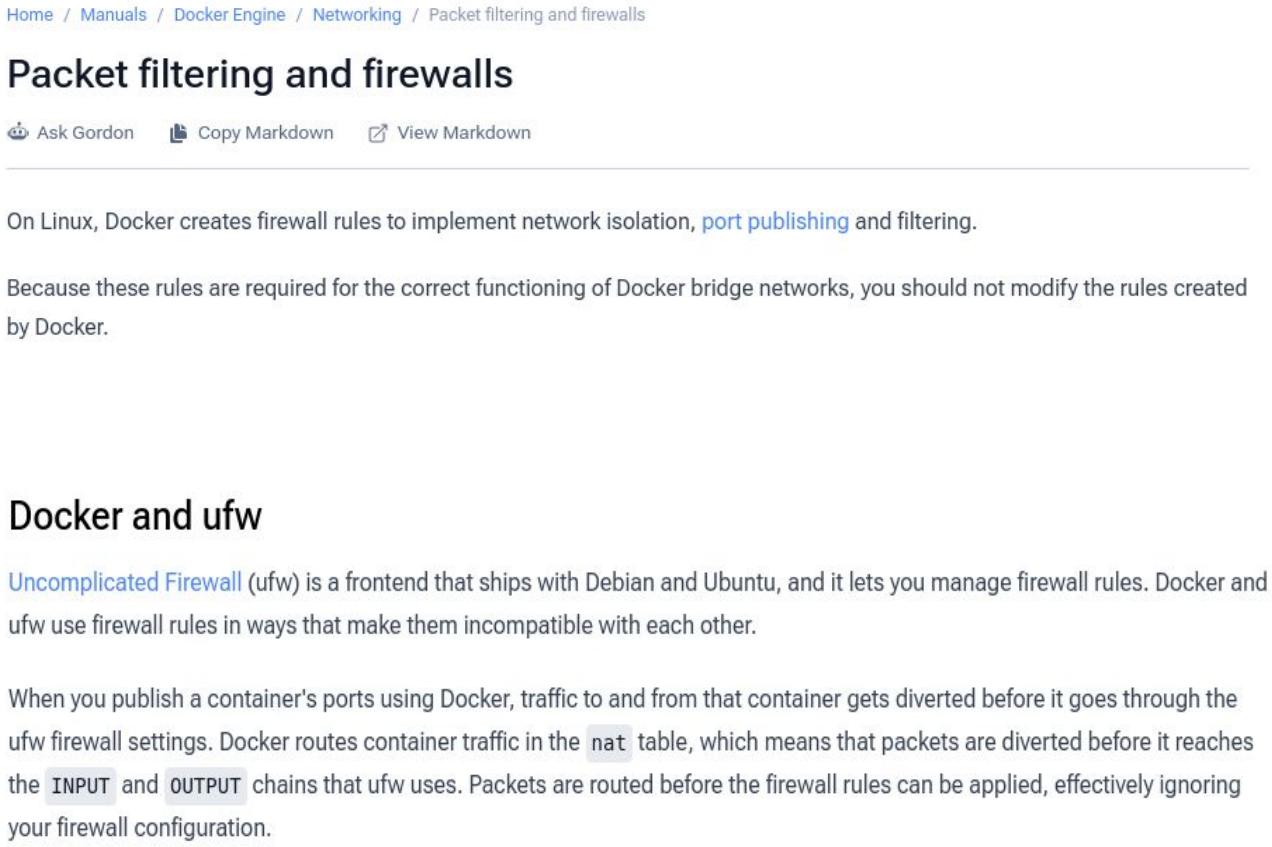


- Typical approach to secure an application in a container:
  - Set up the Dockerfile
  - Double the port blocking by a firewall

A story: Docker and firewalls

YGREKY

- Typical approach to secure an application in a container:
  - Set up the Dockerfile
  - Double the port blocking by a firewall
- Doesn’t work: Docker modifies firewall settings
  - Not visible from ufw
  - Interacts with custom firewall rules
  - Documented: https://docs.docker.com/engine/network/packet-filtering-firewalls/



Home / Manuals / Docker Engine / Networking / Packet filtering and firewalls

Packet filtering and firewalls

Ask Gordon Copy Markdown View Markdown

On Linux, Docker creates firewall rules to implement network isolation, port publishing and filtering.

Because these rules are required for the correct functioning of Docker bridge networks, you should not modify the rules created by Docker.

Docker and ufw

Uncomplicated Firewall (ufw) is a frontend that ships with Debian and Ubuntu, and it lets you manage firewall rules. Docker and ufw use firewall rules in ways that make them incompatible with each other.

When you publish a container's ports using Docker, traffic to and from that container gets diverted before it goes through the ufw firewall settings. Docker routes container traffic in the `nat` table, which means that packets are diverted before it reaches the `INPUT` and `OUTPUT` chains that ufw uses. Packets are routed before the firewall rules can be applied, effectively ignoring your firewall configuration.

Kernel options testing
CONFIG_SECCOMP/CONFIG_SECCOMP_FILTER

YGREKY

Linux kernel default	**Y/Y**
KSPP recommendation	**Y/Y**
Ubuntu	Y/Y
Yocto Project	Y/Y
OpenWRT	Y/Y

Notes on CONFIG_SECCOMP:

- Enables system call filtering (seccomp) with the strict mode (basic syscalls)

Notes on CONFIG_SECCOMP_FILTER:

- Enables seccomp BPF filters

Kernel options testing
CONFIG_SYN_COOKIES

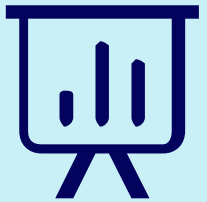
YGREKY

Linux kernel default	**No default**
KSPP recommendation	**Y**
Ubuntu	Y
Yocto Project	Y
OpenWRT	Y

Notes on CONFIG_SYN_COOKIES:

- Denial of service mitigation at the TCP connection creation, adds a challenge before creating connection structures - makes it harder to flood the system with connection attempts

YGREKY



Wrapping up

YGREKY

Research questions - and answers

1. Is it **more likely** to find an option enabled on an embedded platform if it is **enabled by default** in the kernel? -> **it looks so**
2. Are security options set **the same way** on embedded platforms? -> **No...**
3. Do distributions **tend to follow** security recommendations? -> **Partially**

Take-aways

YGREKY

- The default kernel configuration needs changes to be called “secure by default”
  - Distribution work required
  - A warning: custom distributions and the “Operating Systems” category of the CRA
- Security features interacts with applications, or between applications
  - Sometimes in unexpected ways
  - Sometimes no easy fix
- But: well known an enabled by default tend to stay so in distributions
  - Shouldn’t we make more kernel hardening enabled by default?

Questions? Discussion time?





Linux Secure by Default

Are we there yet?

Marta Rybczynska, Ygreky